

# C 언어 EXPRESS(개정4판)



제 14장 포인터 활용

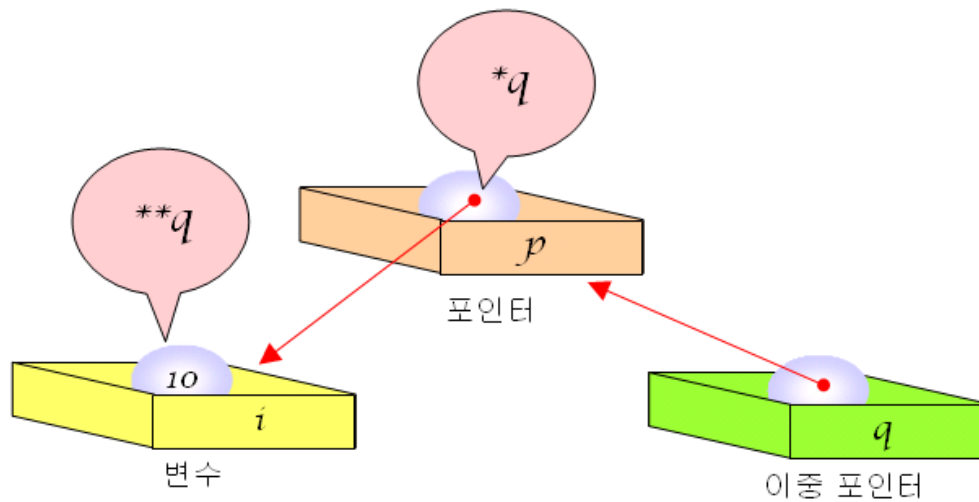


# 이중 포인터

2

- 이중 포인터(double pointer) : 포인터를 가리키는 포인터

```
int i = 10;           // i는 int형 변수  
int *p = &i;          // p는 i를 가리키는 포인터  
int **q = &p;         // q는 포인터 p를 가리키는 이중 포인터
```



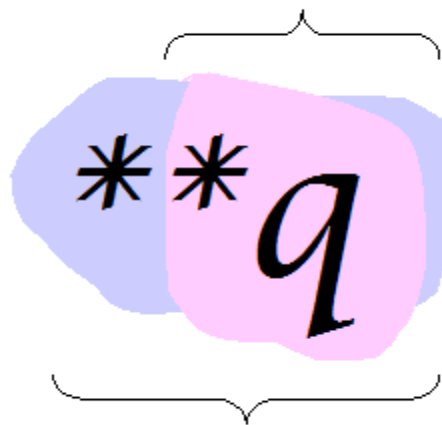


# 이중 포인터

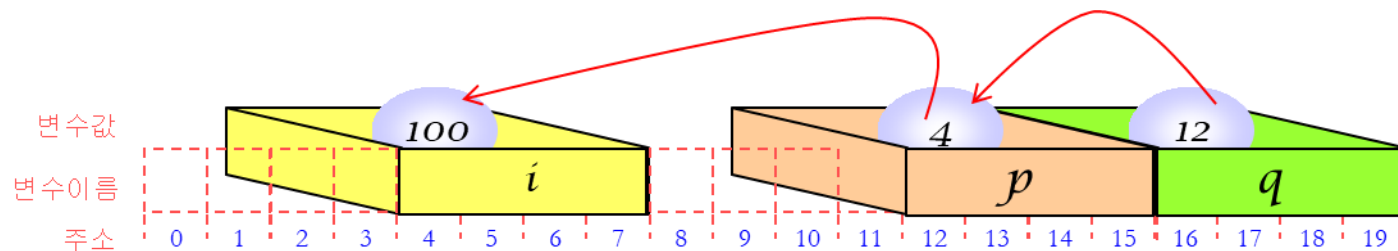
3

- 이중 포인터의 해석

q가 가리키는 위치의 내용



\*q가 가리키는 위치의 내용





# 이중 포인터

4

// 이중 포인터 프로그램

#include <stdio.h>

int main(void)

{

int i = 100;

int\* p = &i;

int\*\* q = &p;

\*p = 200;

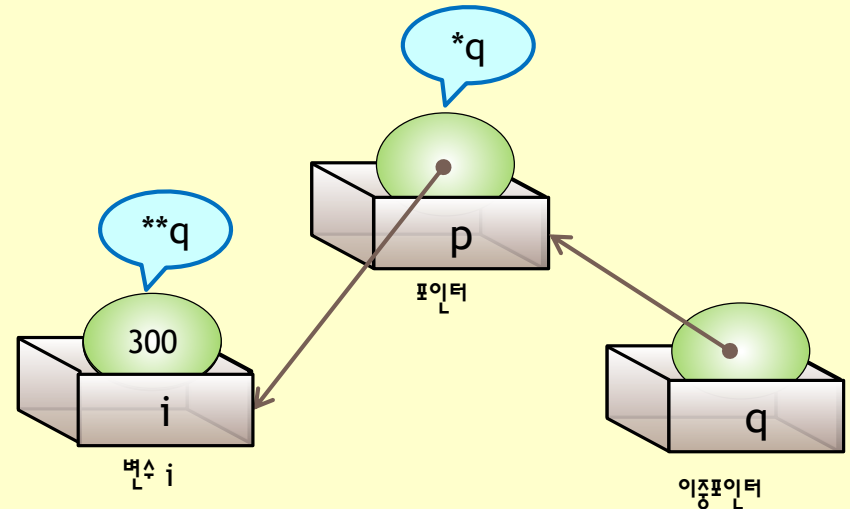
printf("i=%d\n", i);

\*\*q = 300;

printf("i=%d\n", i);

return 0;

}



i=200  
i=300



## 예제 #2

5

```
#include <stdio.h>
```

```
void set_pointer(char** q);
```

```
int main(void)
```

```
{
```

```
    char* p;
```

```
    set_pointer(&p);
```

```
    printf("오늘의 격언: %s \n", p);
```

```
    return 0;
```

```
}
```

```
void set_pointer(char** q)
```

```
{
```

```
    *q = "All that glisters is not gold.";
```

```
}
```

포인터 p의 값을 함수  
에서 변경하려면 주소  
를 보내야 한다.

오늘의 격언: All that glisters is not gold



# 잠모되 코드 라쉬 L

6

```
int main(void)
```

```
{
```

```
    ...
```

```
    char *p;
```

```
    set_pointer(p);
```

```
    ...
```

```
}
```

```
void set_pointer(char *q)
```

```
{
```

```
    q = "All that glisters is not gold.";
```

```
}
```

매개 변수 q만 변경된다.





# 포인터 배열

7

- *포인터 배열(array of pointers)*: 포인터를 모아서 배열로 만든것

① [] 연산자가 \* 연산자보다 우선 순위가 높으므로  
ap는 먼저 배열이 된다.

*int* \* *ap* [10];

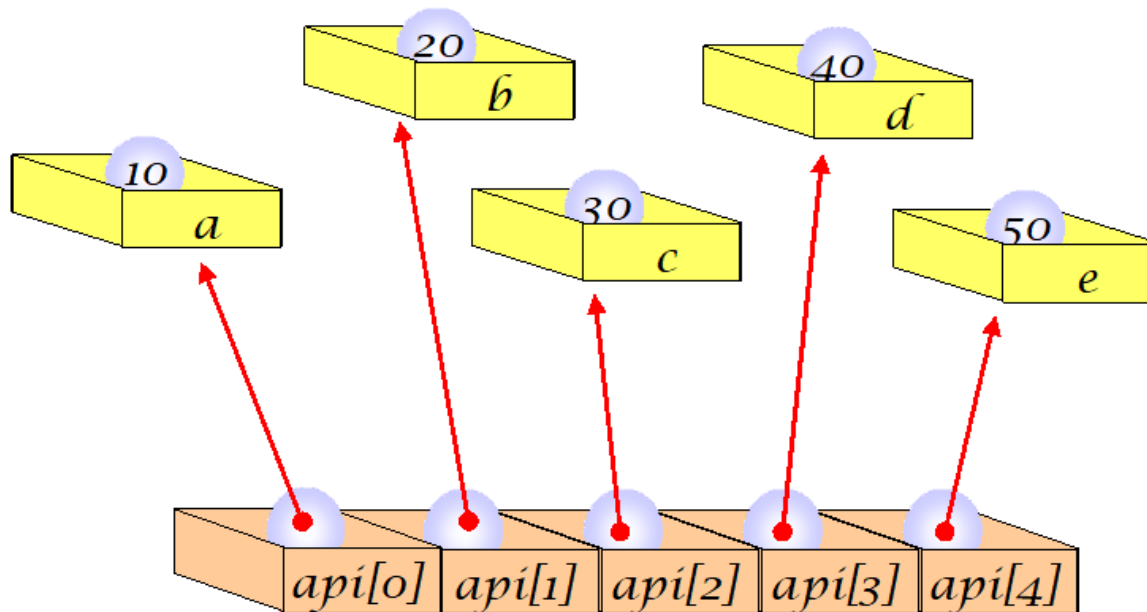
② 어떤 배열이냐 하면 int \*(포인터)들의 배열이 된다.



# 정수형 포인터 배열

8

```
int a = 10, b = 20, c = 30, d = 40, e = 50;  
int *pa[5] = { &a, &b, &c, &d, &e };
```



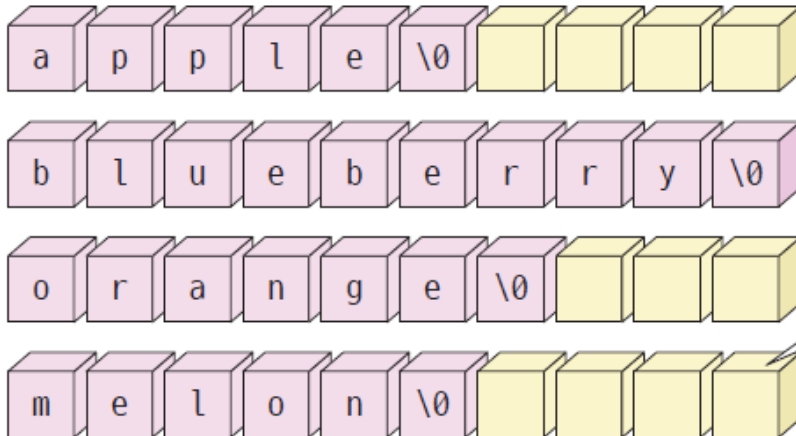




# 2차원 배열에 문자열을 저장

9

```
char fruits[4 ][10] = {  
    "apple",  
    "blueberry",  
    "orange",  
    "melon"  
};
```



2차원 배열을 사용하면  
낭비되는 공간이 생성되죠.

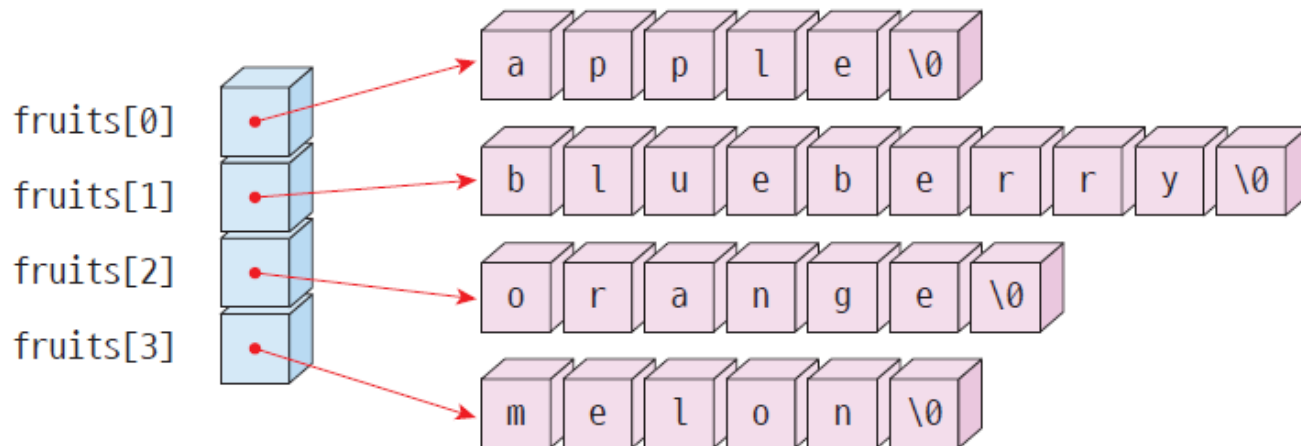




# 문자형 포인터 배열

10

```
char *fruits[ ] = {  
    "apple",  
    "blueberry",  
    "orange",  
    "melon"  
};
```





# 문자열 배열

11

```
#include <stdio.h>

int main(void)
{
    int i, n;
    char *fruits[ ] = {
        "apple",
        "blueberry",
        "orange",
        "melon"
    };

    n = sizeof(fruits)/sizeof(fruits[0]); // 배열 원소 개수 계산

    for(i = 0; i < n; i++)
        printf("%s \n", fruits[i]);

    return 0;
}
```

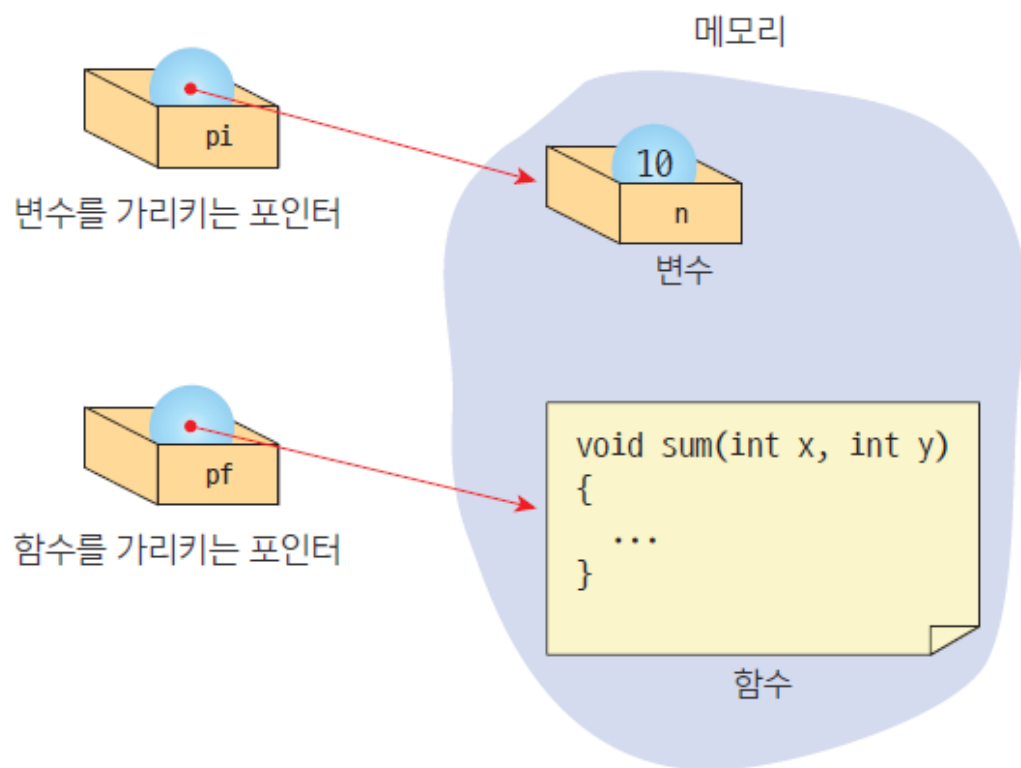
apple  
blueberry  
orange  
melon



# 함수 포인터

12

- 함수 포인터(function pointer): 함수를 가리키는 포인터



함수도 메모리에 코드 형태로 존재하고 따라서 함수도 주소를 가집니다.





# 함수 포인터 정의

13

Syntax

함수 포인터 정의

예

```
int (*pf)(int, int);
```

반드시 괄호가 필요하다. 왜냐하면 괄호에 의하여 pf가 먼저 포인터가 되어야 한다.

함수를 가리키는 포인터를 선언한다.

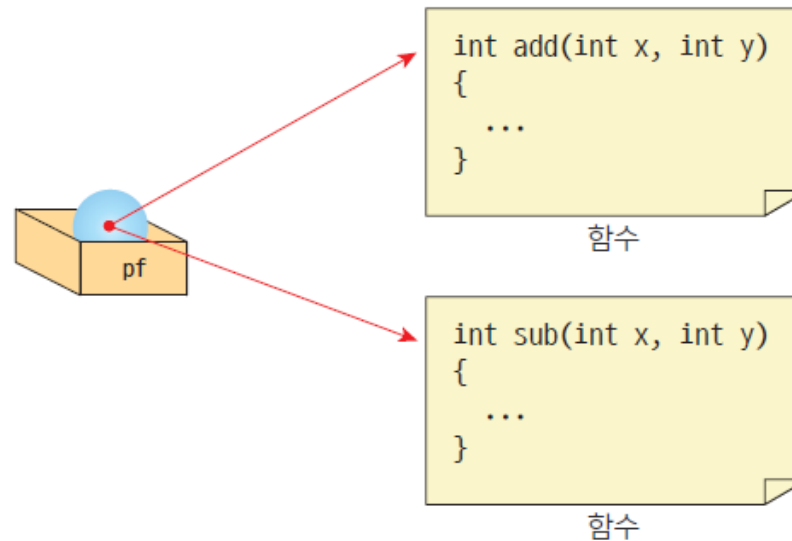
포인터가 가리키는 함수의 매개 변수



# 함수 포인터의 사용

14

```
int add(int, int);           // 함수 원형 정의
int (*pf)(int, int);         // 함수 포인터 정의
...
pf = add;                    // 함수의 이름을 함수 포인터에 대입
result = pf( 10, 20);        // 함수 포인터를 통하여 함수 호출
```





# fp1.c

15

```
#include <stdio.h>
```

```
// 함수 원형 정의
```

```
int add(int, int);
```

```
int sub(int, int);
```

```
int main(void)
```

```
{
```

```
    int result;
```

```
    int (*pf)(int, int);
```

```
// 함수 포인터 정의
```

```
    pf = add;
```

```
    result = pf(10, 20);
```

```
    printf("10+20은 %d\n", result);
```

```
// 함수 포인터에 함수 add()의 주소 대입
```

```
// 함수 포인터를 통한 함수 add() 호출
```

```
    pf = sub;
```

```
    result = pf(10, 20);
```

```
    printf("10-20은 %d\n", result);
```

```
// 함수 포인터에 함수 sub()의 주소 대입
```

```
// 함수 포인터를 통한 함수 sub() 호출
```

```
    return 0;
```

```
}
```



# fp1.c

16

```
int add(int x, int y)
{
    return x+y;
}

int sub(int x, int y)
{
    return x-y;
}
```

10+20은 30  
10-20은 -10





# 함수 포인터의 배열

17

```
int (*pf[5]) (int, int);
```

① [] 연산자가 \* 연산자보다 우선 순위가 높으므로 pf는 먼저 배열이 된다.

*int (\*pf[5]) (int, int);*

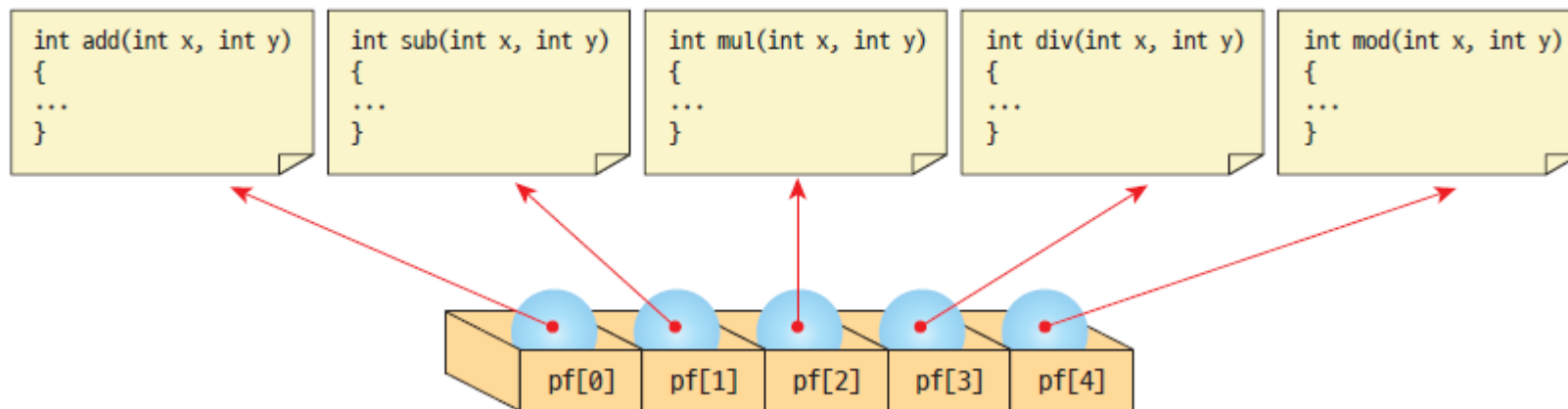
② 어떤 배열이냐 하면 i포인터들의 배열이 된다.

③ 어떤 포인터냐 하면 함수를 가리키는 포인터가 된다.



# 함수 포인터의 배열

18





# 함수 포인터를 이용하여 다음 프로그램을 작성해보자.

19

```
=====
```

- 0. 덧셈
- 1. 뺄셈
- 2. 곱셈
- 3. 나눗셈
- 4. 종료

```
=====
```

메뉴를 선택하시오:2

2개의 정수를 입력하시오:10 20

연산 결과 = 200

```
=====
```

- 0. 덧셈
- 1. 뺄셈
- 2. 곱셈
- 3. 나눗셈
- 4. 종료

```
=====
```

메뉴를 선택하시오:



# 함수 포인터 배열

20

```
// 함수 포인터 배열
```

```
#include <stdio.h>
```

```
// 함수 원형 정의
```

```
void menu(void);
```

```
int add(int x, int y);
```

```
int sub(int x, int y);
```

```
int mul(int x, int y);
```

```
int div(int x, int y);
```

```
void menu(void)
```

```
{
```

```
    printf("=====\n");
```

```
    printf("0. 덧셈\n");
```

```
    printf("1. 뺄셈\n");
```

```
    printf("2. 곱셈\n");
```

```
    printf("3. 나눗셈\n");
```

```
    printf("4. 종료\n");
```

```
    printf("=====\n");
```

```
}
```



## 하스 포인터 배열

21

```
int main(void)
{
    int choice, result, x, y;
    // 함수 포인터 배열을 선언하고 초기화한다.
    int (*pf[4])(int, int) = { add, sub, mul, div };

    while(1)
    {
        menu();
        printf("메뉴를 선택하시오:");
        scanf("%d", &choice);

        if( choice < 0 || choice >=4 )
            break;
        printf("2개의 정수를 입력하시오:");
        scanf("%d %d", &x, &y);

        result = pf[choice](x, y);    // 함수 포인터를 이용한 함수 호출
        printf("연산 결과 = %d\n", result);
    }
    return 0;
}
```



# 함수 포인터 배열

22

```
int add(int x, int y)
{
    return x + y;
}

int sub(int x, int y)
{
    return x - y;
}

int mul(int x, int y)
{
    return x * y;
}

int div(int x, int y)
{
    return x / y;
}
```

=====

- 0. 덧셈
- 1. 뺄셈
- 2. 곱셈
- 3. 나눗셈
- 4. 종료

=====

메뉴를 선택하시오:2  
2개의 정수를 입력하시오:10 20  
연산 결과 = 200

=====

- 0. 덧셈
- 1. 뺄셈
- 2. 곱셈
- 3. 나눗셈
- 4. 종료

=====

메뉴를 선택하시오:



# 함수 인수로서의 함수 포인터

23

- 함수 포인터도 인수로 전달이 가능하다.

특정 함수를 호출하게  
할 수 있어요.



```
int main(void)
{
    ...
    sub(f);
    ...
}
```

```
int sub( void (*pf)() )
{
    ...
    pf();
    ...
}
```

```
void f()
{
    ...
}
```



## 예제

24

- 다음과 같은 수식을 계산하는 프로그램을 작성하여 보자.

$$\sum_1^n (f^2(k) + f(k) + 1)$$

- 여기서  $f(k)$ 는 다음과 같은 함수들이 될 수 있다.

$$f(k) = \frac{1}{k} \quad \text{또는} \quad f(k) = \cos(k)$$





# 예제

25

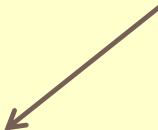
```
#include <stdio.h>
#include <math.h>

double f1(double k);
double f2(double k);
double formula(double (*pf)(double), int n);

int main(void)
{
    printf("%f\n", formula(f1, 10));
    printf("%f\n", formula(f2, 10));
}

double formula(double (*pf)(double), int n)
{
    int i;
    double sum = 0.0;

    for(i = 1; i < n; i++)
        sum += pf(i) * pf(i) + pf(i) + 1;
    return sum;
}
```

$$\sum_{k=1}^n (f^2(k) + f(k) + 1)$$




# 예제

26

```
double f1(double k)
{
    return 1.0 / k;
}

double f2(double k)
{
    return cos(k);
}
```

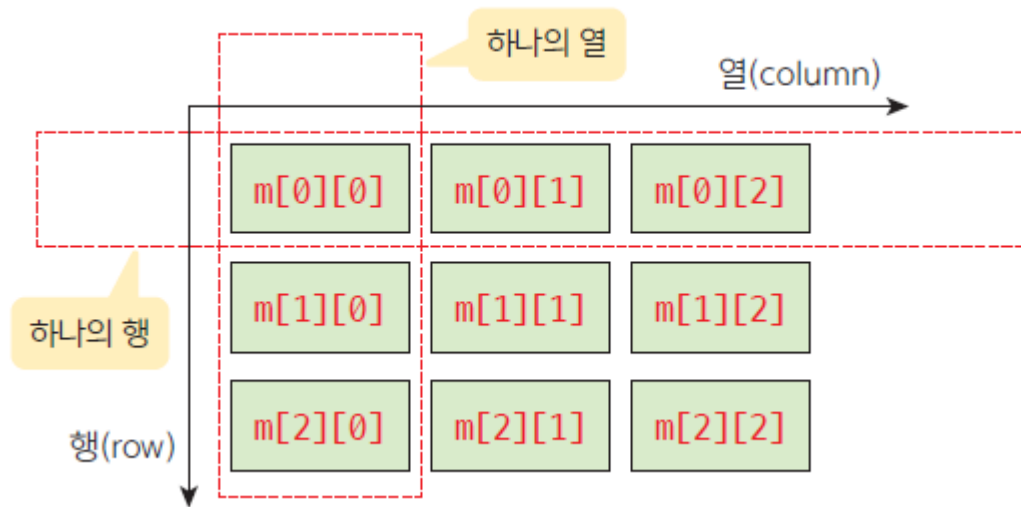
13.368736  
12.716152



# 다차원 배열과 포인터

27

- 2차원 배열 `int m[3][3]`
- 1행->2행->3행->...순으로 메모리에 저장(행우선 방법)

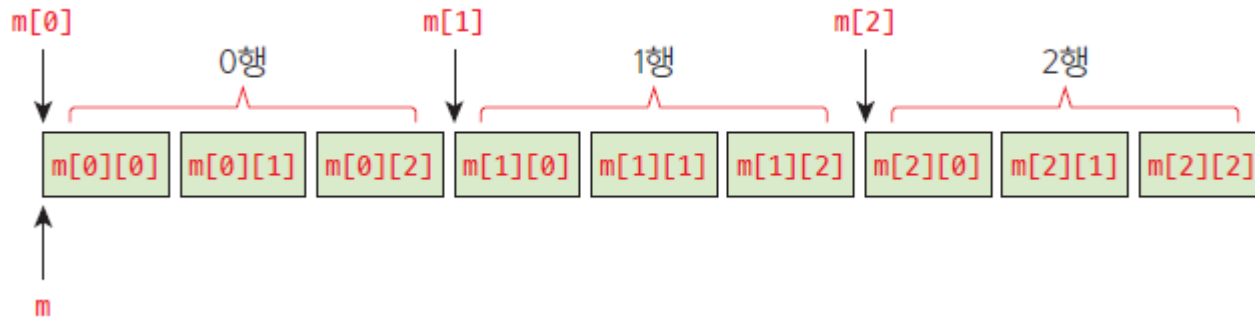




# 행우선 저장 방법

28

- 행우선 방법(row-major)으로 행을 기준으로 하여서 2차원 배열을 메모리에 저장한다. 즉 0번째 행을 먼저 저장한 후에, 1번째 행에 속하는 요소들을 저장하는 방법이다.

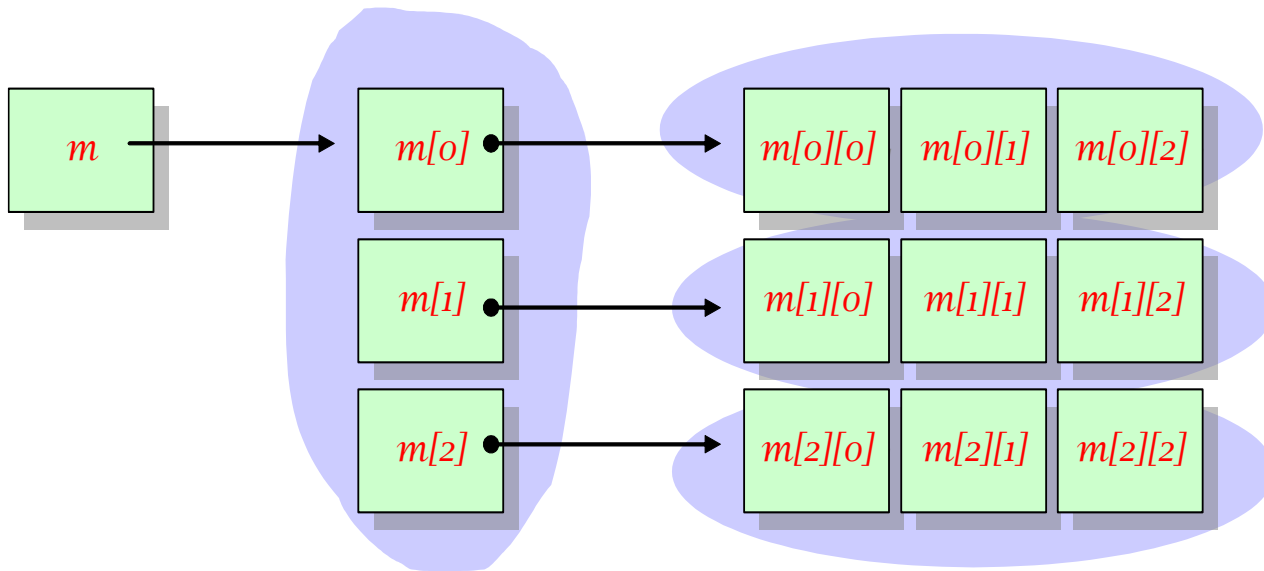




# 2차원 배열과 포인터

29

- 배열 이름  $m$ 은  $\&m[0][0]$
- $m[0]$ 는 1행의 시작 주소
- $m[1]$ 은 2행의 시작 주소
- ...





# 2차원 배열의 해석

30

m은 세개의 원소를  
가지는 배열이다.

```
int m[3][3];
```

해석의 방향

그 원소들은 다시 세  
개의 원소로 되어 있  
다.



# 포인터를 이용한 배열 요소 방문

31

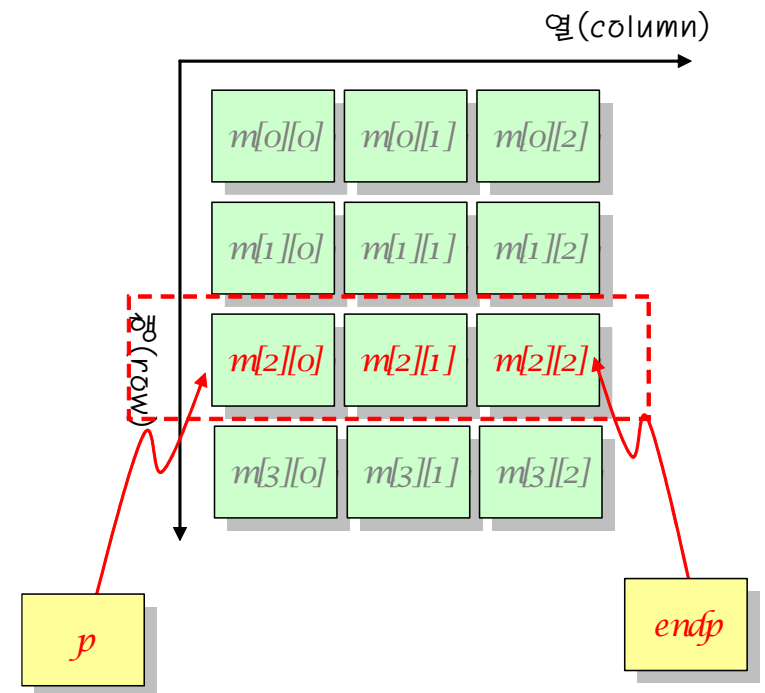
- 행의 평균을 구하는 경우

```
double get_row_avg(int m[][COLS], int r)
{
    int *p, *endp;
    double sum = 0.0;

    p = &m[r][0];
    endp = &m[r][COLS-1];

    while( p <= endp )
        sum += *p++;

    sum /= COLS;
    return sum;
}
```





# 포인터를 이용한 배열 요소 방문

32

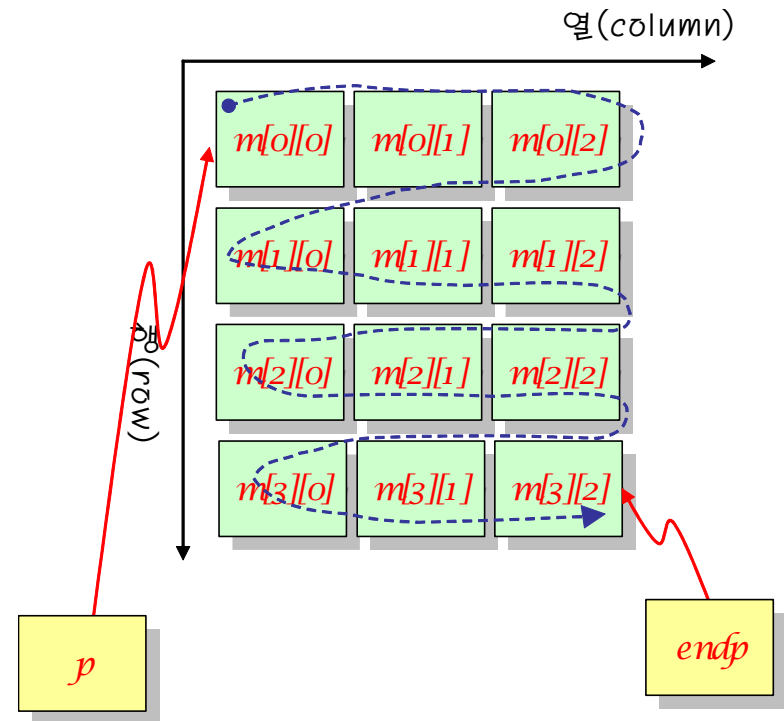
- 전체 원소의 평균을 구하는 경우

```
double get_total_avg(int m[][COLS])
{
    int *p, *endp;
    double sum = 0.0;

    p = &m[0][0];
    endp = &m[ROWS-1][COLS-1];

    while( p <= endp )
        sum += *p++;

    sum /= ROWS * COLS;
    return sum;
}
```







# const 포인터

33

- const를 붙이는 위치에 따라서 의미가 달라진다.

p가 가리키는 내용이 변경되지 않음을 나타낸다.

*const char \*p;*

포인터 p가 변경되지 않음을 나타낸다.

*char \*const p;*



# 예제

34

```
#include <stdio.h>
int main(void)
{
    char s[] = "Barking dogs seldom bite.";
    char t[] = "A bad workman blames his tools";
    const char * p=s;
    char * const q=s;

    //p[3] = 'a';
    p = t;
    q[3] = 'a';
    //q = t;

    return 0;
}
```

p가 가리키는 곳의 내용을 변경할 수 없다.

하지만 p는 변경이 가능하다.

q가 가리키는 곳의 내용은 변경할 수 있다.

하지만 q는 변경이 불가능하다.



# volatile 포인터

35

- **volatile**은 다른 프로세스나 스레드가 값을 항상 변경할 수 있으니 값을 사용할 때마다 다시 메모리에서 읽으라는 것을 의미

p가 가리키는 내용이 수시로 변경되니 사용할 때마다 다시 로드하라는 의미이다.



```
volatile char *p;
```



# void 포인터

36

- 순수하게 메모리의 주소만 가지고 있는 포인터(generic pointer)
- 가리키는 대상물은 아직 정해지지 않음  
(예) `void *vp;`
- 다음과 같은 연산은 모두 오류이다.

```
*vp;           // 오류  
vp++;          // 오류  
vp--;          // 오류
```



# void 포인터는 어디에 사용하는가?

37

- void 포인터를 이용하면 어떤 타입의 포인터도 받을 수 있는 함수를 작성할 수 있다. 예를 들어서 전달받은 메모리를 0으로 채우는 함수를 작성해보면 다음과 같다.

```
void memzero(void *ptr, size_t len)
{
    char* p = (char*)ptr;
    for (; len > 0; len--)
        *p++ = 0;
}
```



```
#include <stdio.h>
void memzero(void *ptr, size_t len)
{
    char* p = (char*)ptr;
    for (; len > 0; len--)
        *p++ = 0;
}

int main(void)
{
    char a[10];
    memzero(a, sizeof(a));

    int b[10];
    memzero(b, sizeof(b));

    double c[10];
    memzero(c, sizeof(c));

    return 0;
}
```



# main() 함수의 인수

39

- 지금까지의 main() 함수 형태

```
int main(void)
{
  ..
}
```

- 외부로부터 입력을 받는 main() 함수 형태

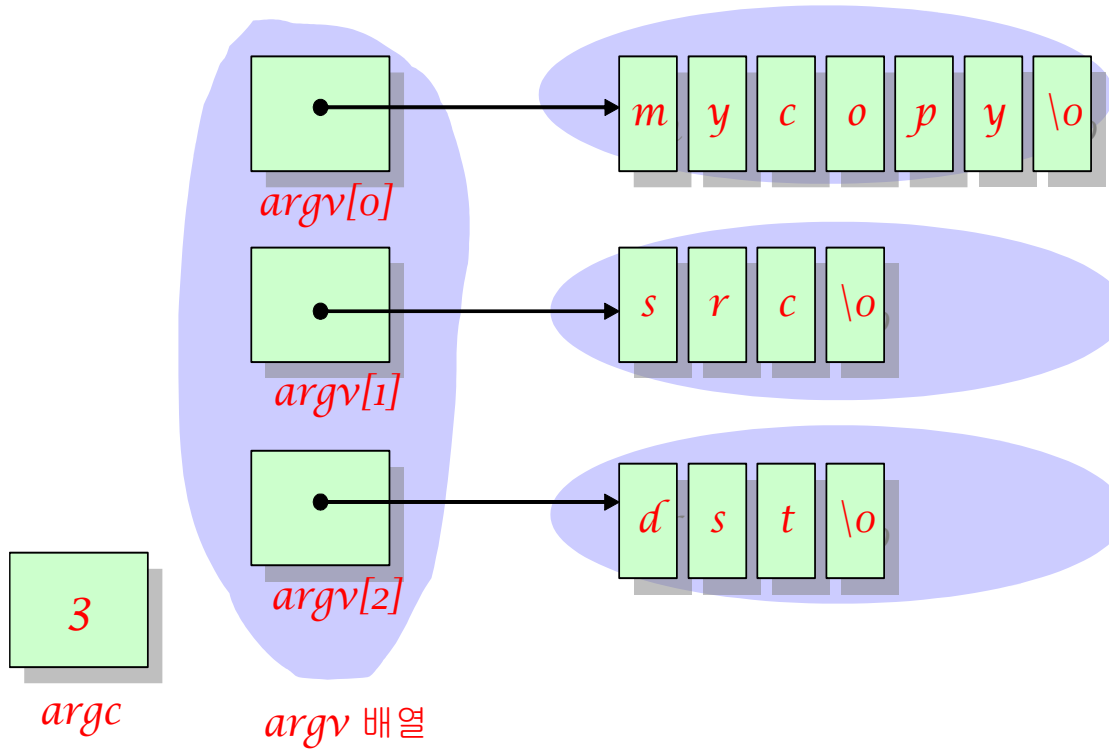
```
int main(int argc, char *argv[])
{
  ..
}
```



# 인수 전달 방법

40

```
C: \cprogram> mycopy src dst
```







# main\_arg.c

41

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i = 0;

    for(i = 0; i < argc; i++)
        printf("명령어 라인에서 %d번째 문자열 = %s\n", i, argv[i]);

    return 0;
}
```

```
c:\cprogram\mainarg\Debug>mainarg src dst
명령어 라인에서 0번째 문자열 = mainarg
명령어 라인에서 1번째 문자열 = src
명령어 라인에서 2번째 문자열 = dst
c:\cprogram\mainarg\Debug>
```



# Lab: 프로그램 인수 사용하기

42

- 사용자로부터 마일(mile)로 된 거리를 받아서 킬로미터(km)로 변환해주는 프로그램을 작성해보자.

```
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All right reserved.
e:>mile2km 10
입력된 거리는 16.090000 km입니다.
e:>
```



# mile2km.c

43

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    double mile, km;

    if( argc != 2 ){
        printf("사용 방법: mile2km 거리\n");
        return 1;
    }
    mile = atof(argv[1]);
    km = 1.609 * mile;
    printf("입력된 거리는 %f km입니다. \n", km);

    return 0;
}
```



# Q & A

44

